

Monitoring and Alerting Setup

wiki/sre/monitoring-setup · Updated 2023-01-20

Owner: Site Reliability Engineering · POC: Dani Okafor, SRE Lead

Overview

This page explains how observability is set up for Northwind services and how to get access to dashboards and alerts. It covers metrics, logs, host checks, and alert routing. It is maintained by the SRE team.

Good monitoring is what turns an outage from a customer-reported surprise into something we catch and fix first. Every production service is expected to emit the standard metrics described below.

The four golden signals

Our monitoring is built around four signals. Every service dashboard shows all four, and most alerts are derived from them.

- Latency: how long requests take, measured at several percentiles.
- Traffic: how much demand the service is handling.
- Errors: the rate of failed requests.
- Saturation: how full the service's resources are.

Metrics and dashboards

We monitor all services using Stackdriver. Dashboards live in the Stackdriver console. Each service has a standard dashboard showing request rate, error rate, latency percentiles, and saturation. Teams may add service-specific panels, but the four standard signals must always be present.

- Request rate: requests per second, split by endpoint and status class.
- Error rate: the percentage of 5xx responses over a rolling window.
- Latency: p50, p95, and p99 response times.
- Saturation: CPU, memory, and connection-pool usage.

Custom metrics

For custom metrics, export to Stackdriver Monitoring via the agent. Keep metric cardinality under control: avoid high-cardinality labels such as raw user IDs, which make dashboards slow and expensive. Prefer a small, fixed set of label values.

Dashboard conventions

Keep dashboards readable. Put the four golden signals at the top, group related panels, and label axes with units. A dashboard that nobody can interpret during an incident is worse than no dashboard at all.

Logs

Structured logs complement metrics. Emit logs as structured records with a consistent set of fields so they can be searched and correlated.

- Include a request identifier so logs can be traced across services.
- Log at an appropriate level; do not log sensitive data.
- Keep log volume reasonable to control cost and noise.

Application logs are retained according to the data retention policy. Do not rely on logs for long-term storage of anything you need to keep.

Host checks

Host-level checks run on the Nagios box at `nagios.corp.internal`; ping the SRE team if you need an account. These checks cover disk usage, process liveness, and certificate expiry on the legacy hosts that are not yet fully migrated. New services on managed platforms do not need Nagios checks and should rely on platform health checks instead.

Legacy hosts

A handful of legacy hosts are still monitored this way while they are being migrated. Once a service moves to a managed platform, its Nagios checks are removed and platform health checks take over.

Alerting

Alerts are routed to the `#alerts` channel. Every alert must be actionable and must link to a runbook. Alerts that fire repeatedly without a clear action are noise and should be tuned or removed.

- Page-level alerts notify the on-call engineer directly and are reserved for customer impact.

- Ticket-level alerts create a tracked item for the owning team to handle during business

hours.

- Informational alerts post to the channel only and never page.

Alert hygiene

Review alert volume monthly. If an alert has not led to a useful action in the last quarter, it is a candidate for removal. The goal is that every page means something is actually wrong. Alert fatigue is dangerous: when every page is noise, the one real page gets ignored.

Writing a good alert

A good alert fires on customer-visible symptoms, not on internal causes. Alert on high error rate or latency, not on a single server being busy. Always link the alert to a runbook that tells the responder what to do.

Worked example

A service's error rate crosses its threshold. A page-level alert fires to the on-call engineer and posts to the #alerts channel with a link to the service runbook. The engineer opens the Stackdriver dashboard, sees error rate and latency both climbing after a recent deploy, rolls the deploy back, watches the signals recover, and closes the alert. The runbook link made the first thirty seconds obvious.

Getting access

To get dashboard and alerting access, open a request with the SRE team and include the services you work on. Access is granted by team and reviewed periodically. If you are on-call, make sure you can log in and silence alerts before your rotation begins.

Frequently asked questions

Why is my dashboard slow? Usually high-cardinality labels. Reduce the number of distinct label values.

How do I silence a noisy alert during an incident? Silence it from the alerting console for a bounded time, and follow up to tune or remove it.

Do new services need Nagios checks? No. Rely on platform health checks; Nagios is for legacy hosts only.

Common mistakes

- Alerting on causes instead of customer-visible symptoms.
- Creating alerts with no linked runbook.
- Logging sensitive data or logging so much that signal is lost.
- Leaving stale alerts firing until everyone ignores the channel.

Responsibilities

Service teams own the thresholds and runbooks for their own alerts. The SRE team owns the shared monitoring platform, the standard dashboard

template, and the alert routing configuration.

Related documents

See the On-Call Runbook for how pages are handled, the Incident Severity Guide for classifying impact, and the Data Retention Policy for how long logs are kept.

Service level objectives

A service level objective expresses, as a target, how reliable a service should be from the customer's point of view, for example a percentage of requests served successfully and quickly over a window. Objectives turn vague reliability goals into something measurable that teams can act on.

- Define objectives around customer-visible behaviour, not internal implementation details.
- Track the error budget: the amount of unreliability the objective allows before action is required.
- When the error budget is nearly spent, prioritise reliability work over new features.
- Review objectives periodically so they still reflect what customers actually need.

Alerts derived from these objectives tend to be far better than ad-hoc thresholds, because they fire on real customer impact rather than on internal noise.